Penjelasan Lengkap

Kalkulator Permutasi & Kombinasi

Program Python interaktif dengan library *rich*
Menghitung rumus + menampilkan seluruh susunan lengkap

Dokumen ini adalah bahan belajar untuk memahami file **main.py**: konsep matematikanya, arti setiap baris kode, alur program, contoh jalannya, serta catatan perbaikan.

Bagian	Isi
1	Gambaran umum program
2	Dasar teori: permutasi dan kombinasi
3	Penjelasan kode bagian per bagian
4	Diagram alur
5	Contoh jalannya program
6	Catatan penting dan ide pengembangan
7	Ringkasan cepat dan latihan pemahaman

1. Gambaran Umum Program

Program ini adalah aplikasi terminal interaktif dengan dua kemampuan utama:

- **Menghitung jumlah** permutasi P(n, r) dan kombinasi C(n, r) memakai rumus.
- **Menampilkan seluruh susunannya** dalam tabel yang rapi, bukan hanya angka totalnya.

Library	Fungsi
`math`	Menghitung jumlah permutasi/kombinasi (`math.perm`, `math.comb`)
`itertools`	Menghasilkan semua susunan (`permutations`, `combinations`)
`sys`	Cek sistem operasi dan keluar dari program (`sys.exit`)
`typing`	Petunjuk tipe data (`List`, `Any`)
`rich`	Tampilan terminal berwarna: panel, tabel, dan prompt input

2. Dasar Teori Matematika

Permutasi (urutan diperhatikan)

Memilih **r** elemen dari **n** elemen, dan urutan berpengaruh (AB ≠ BA).

P(n, r) = n! / (n − r)!

Kombinasi (urutan tidak diperhatikan)

Memilih **r** elemen dari **n** elemen, dan urutan tidak berpengaruh (AB = BA).

C(n, r) = n! / (r! × (n − r)!)

Contoh: elemen {A, B, C} dengan r = 2

Jenis	Perhitungan	Hasil
Permutasi	P(3,2) = 3! / (3−2)! = 6 / 1	**6 cara** (A,B) (A,C) (B,A) (B,C) (C,A) (C,B)
Kombinasi	C(3,2) = 3! / (2! × 1!) = 6 / 2	**3 cara** (A,B) (A,C) (B,C)

Perhatikan bahwa (A,B) dan (B,A) dianggap **berbeda** pada permutasi, tetapi **sama** pada kombinasi. Itu sebabnya jumlah permutasi selalu lebih besar atau sama dengan kombinasi. Hubungan keduanya: **P(n,r) = C(n,r) × r!**

3. Penjelasan Kode Bagian per Bagian

3.1 Import dan pengaturan awal

```
import math
import itertools
import sys
from typing import List, Any
```

Mengimpor semua modul yang dibutuhkan. `List` dipakai untuk menulis tipe, misalnya `List[str]` yang berarti "daftar berisi string".

3.2 Perbaikan encoding UTF-8 di Windows

```
if sys.platform == "win32":
    try:
        if sys.stdout.encoding.lower() != "utf-8":
            sys.stdout.reconfigure(encoding="utf-8")
        ...
    except Exception:
        pass
```

- `sys.platform == "win32"` memeriksa apakah program berjalan di Windows.
- Terminal Windows kadang tidak memakai UTF-8, sehingga karakter khusus (garis kotak tabel dari *rich*) bisa rusak.
- `reconfigure(encoding="utf-8")` memaksa output memakai UTF-8.
- `try/except ... pass` membuat program tetap jalan walaupun pengaturan ini gagal.

3.3 Import rich dengan pengaman

```
try:
    from rich.console import Console
    ...
except ImportError:
    print("[ERROR] Library 'rich' belum terpasang.")
    sys.exit(1)
```

Jika *rich* belum diinstal, program tidak crash dengan error panjang. Ia menampilkan pesan ramah lalu berhenti (`sys.exit(1)`, kode 1 berarti keluar karena error). Cara menginstal: `pip install rich`.

Komponen rich	Fungsi
`Console`	Pencetak ke terminal (pengganti `print`) yang mendukung warna
`Table`	Membuat tabel
`Panel`	Kotak berbingkai
`Text`	Teks yang bisa diberi gaya
`Prompt / IntPrompt / Confirm`	Input teks, input angka (otomatis divalidasi), dan input ya/tidak
`box`	Pilihan gaya garis bingkai

3.4 Fungsi print_header()

```
title_text = Text("KALKULATOR PERMUTASI & KOMBINASI", style="bold cyan")
subtitle_text = Text("Hitung Rumus + ...", style="italic yellow")
panel = Panel(
    Text.assemble(title_text, "\n", subtitle_text, justify="center"),
    box=box.DOUBLE_EDGE, border_style="bright_blue", padding=(1, 2),
)
console.print(panel)
```

Membuat judul aplikasi: judul tebal cyan dan subjudul miring kuning. `Text.assemble` menggabungkan keduanya dengan baris baru, diratakan ke tengah (`justify="center"`), lalu dibungkus `Panel` bergaris ganda dengan jarak dalam (*padding*) 1 baris vertikal dan 2 spasi horizontal.

3.5 Fungsi input_elements(): meminta elemen dari pengguna

Fungsi ini mengembalikan daftar elemen (`List[str]`) dengan dua mode.

Mode 1: ketik elemen sendiri

```
raw_input = Prompt.ask("Masukkan elemen (pisahkan dengan koma atau spasi)")
if "," in raw_input:
    elements = [item.strip() for item in raw_input.split(",") if item.strip()]
else:
    elements = [item.strip() for item in raw_input.split() if item.strip()]
```

- Jika ada koma, dipisah berdasarkan koma. Ini memungkinkan elemen yang mengandung spasi, misalnya *Apel Merah, Jeruk*.
- Jika tidak ada koma, dipisah berdasarkan spasi.
- `.strip()` membuang spasi di awal dan akhir.
- `if item.strip()` membuang elemen kosong, misalnya dari input *A,,B*.
- Jika hasilnya kosong, dipakai nilai default `["A", "B", "C"]`.
- Teknik `[... for ... in ... if ...]` ini disebut **list comprehension**.

Mode 2: hanya masukkan n

```
n = IntPrompt.ask("Masukkan nilai n", default=4)
while n <= 0:
    ...
if n <= 26:
    elements = [chr(65 + i) for i in range(n)]
else:
    elements = [str(i + 1) for i in range(n)]
```

- `IntPrompt` memastikan input berupa bilangan bulat.
- `while n <= 0` mengulang permintaan sampai n valid.

- `chr(65)` = "A", `chr(66)` = "B", dan seterusnya (kode ASCII). Jadi n=4 menghasilkan `['A','B','C','D']`.
- Jika n lebih dari 26 (huruf habis), dipakai angka '1','2','3', dan seterusnya.

3.6 Fungsi display_arrangements(): menampilkan tabel susunan

Ini bagian paling kompleks. Alurnya terdiri dari enam langkah.

Langkah 1: cek jumlah data

```
total = len(items)
if total == 0: ... return
```

Langkah 2: batasi tampilan jika terlalu banyak

```
if total > max_display:   # max_display = 200
    show_all = Confirm.ask("... Tampilkan semua?", default=False)
    if not show_all:
        display_items = items[:max_display]
        is_truncated = True
```

Jika lebih dari 200 susunan, pengguna ditanya apakah mau menampilkan semuanya. Default-nya "tidak", sehingga hanya 200 pertama yang ditampilkan agar terminal tidak membeku.

Langkah 3: hitung jumlah kolom otomatis

```
sample_items = [", ".join(map(str, x)) for x in display_items[:50]]
max_item_len = max((len(s) for s in sample_items), default=6) + 4
unit_width = max(max_item_len + 8, 18)
term_width = console.width if console.width else 80
num_columns = max(1, min(4, term_width // unit_width))
```

- Setiap susunan (tuple) diubah menjadi teks, misalnya ('A','B') menjadi "A, B".
- Diambil panjang teks terpanjang dari 50 sampel pertama, ditambah ruang cadangan.
- `unit_width` adalah lebar satu pasangan kolom (No. + Susunan), minimal 18 karakter.
- `term_width // unit_width` menghitung berapa pasangan kolom muat di lebar terminal, lalu dibatasi antara **1 sampai 4 kolom**.

Dengan begitu tampilan menyesuaikan lebar terminal dan tidak menjadi satu kolom panjang ke bawah.

Langkah 4: buat tabel dan kolomnya

```
for col_idx in range(num_columns):
    table.add_column("No.", justify="right", ...)
    table.add_column("Susunan", justify="center", ...)
```

Setiap "blok kolom" terdiri dari dua kolom: nomor dan susunan.

Langkah 5: susun data ke baris

```
for idx, item in enumerate(display_items, start=1):
    formatted_item = ", ".join(map(str, item))
    current_row.extend([f"{idx}.", f"({formatted_item})"])
    if len(current_row) == num_columns * 2:
        rows.append(current_row)
        current_row = []
```

- `enumerate(..., start=1)` memberi nomor mulai dari 1.
- Setiap item menambah 2 sel: nomor (misalnya "1.") dan susunan (misalnya "(A, B)").
- Saat satu baris penuh (jumlah sel = kolom × 2), baris disimpan dan dimulai baris baru.
- Sisa item yang belum memenuhi baris diisi sel kosong agar tabel tetap rata.

Contoh dengan 3 kolom dan 7 item:

```
1. (A,B)    2. (A,C)    3. (A,D)
4. (B,A)    5. (B,C)    6. (B,D)
7. (C,A)    (kosong)    (kosong)
```

Langkah 6: cetak tabel dan catatan

Jika data dipotong, ditampilkan keterangan "Menampilkan 200 dari N susunan".

3.7 Fungsi process_permutation(elements, r)

```
n = len(elements)
total_perm = math.perm(n, r)
...
arrangements = list(itertools.permutations(elements, r))
display_arrangements(f"Daftar ... P({n}, {r})", arrangements)
```

- `math.perm(n, r)` menghitung P(n, r) langsung (tersedia mulai Python 3.8).
- Ditampilkan **panel ringkasan** berisi definisi, himpunan elemen, nilai r, rumus, dan hasil perhitungan.
- Tanda `{{{...}}}` di f-string menghasilkan kurung kurawal literal dengan isi variabel di dalamnya.
- `itertools.permutations(elements, r)` menghasilkan semua urutan r elemen.

Contoh: `permutations(['A','B','C'], 2)` memberi (A,B), (A,C), (B,A), (B,C), (C,A), (C,B).

3.8 Fungsi process_combination(elements, r)

Strukturnya sama persis, tetapi memakai `math.comb(n, r)` dan `itertools.combinations(elements, r)`. Contoh: `combinations(['A','B','C'], 2)` memberi (A,B), (A,C), (B,C). Panel berwarna kuning agar mudah dibedakan dari permutasi yang berwarna cyan.

3.9 Fungsi main(): alur utama program

```
while True:
    console.clear()
    print_header()
    ... tampilkan menu ...
    choice = Prompt.ask("Pilih opsi", choices=["1","2","3","4"], default="1")
```

Loop `while True` membuat program berjalan terus sampai pengguna memilih keluar. Alur satu putaran:

- Layar dibersihkan, header dan menu ditampilkan.
- Jika memilih **4**, program berpamitan lalu `break` (keluar).
- Jika tidak, panggil `input_elements()` untuk mendapat elemen, lalu hitung `n = len(elements)`.
- Minta nilai **r** dengan validasi: `while r < 1 or r > n` mengulang sampai r berada di rentang 1..n. Default r adalah `min(2, n)`, sehingga jika n=1 defaultnya 1.
- Sesuai pilihan menu: 1 menjalankan permutasi, 2 kombinasi, 3 keduanya (komparasi).
- Pengguna ditanya apakah ingin menghitung lagi. Jika tidak, program berakhir.

3.10 Penutup

```
if __name__ == "__main__":
    main()
```

Artinya `main()` hanya dijalankan jika file ini dieksekusi langsung (`python main.py`), bukan saat file di-*import* dari file lain.

4. Diagram Alur Singkat

```
Mulai
  ↓
Tampilkan header + menu
  ↓
Pilih 1/2/3/4 ──(4)──→ Keluar
  ↓ (1/2/3)
Input elemen (manual / otomatis dari n)
  ↓
Input r (divalidasi 1..n)
  ↓
Hitung jumlah (math.perm / math.comb)
  ↓
Tampilkan panel ringkasan
  ↓
Buat semua susunan (itertools)
  ↓
Tampilkan tabel (maks. 200 kecuali diminta semua)
  ↓
Ulangi? ──(ya)──→ kembali ke menu
  ↓ (tidak)
Selesai
```

5. Contoh Jalannya Program

Input: menu **3** (keduanya), elemen `A, B, C`, r = **2**.

Jenis	Perhitungan	Hasil
Permutasi	P(3,2) = 3! / (3−2)! = 6	**6 cara**: (A,B) (A,C) (B,A) (B,C) (C,A) (C,B)
Kombinasi	C(3,2) = 3! / (2! × 1!) = 3	**3 cara**: (A,B) (A,C) (B,C)

6. Catatan Penting dan Ide Pengembangan

Bagian ini berguna kalau ditanya dosen atau ingin menambah nilai tugas.

1. Jumlah susunan bisa meledak. P(10,10) = 3.628.800 dan P(26,10) lebih dari 19 triliun. Di kode ini `list(itertools.permutations(...))` dijalankan **sebelum** pengguna ditanya mau menampilkan semua atau tidak, sehingga untuk n besar program bisa lambat atau kehabisan memori sebelum sempat bertanya. Perbaikan: cek `total_perm` dulu, dan jika besar ambil sebagian saja dengan `itertools.islice(itertools.permutations(elements, r), 200)` tanpa membuat seluruh list.

2. Elemen duplikat dianggap berbeda. Input *A, A, B* diperlakukan sebagai tiga elemen berbeda, sehingga hasilnya tidak sesuai rumus permutasi dengan unsur sama. Bisa diperbaiki dengan menolak atau menghapus duplikat (`dict.fromkeys(elements)`).

3. Kurung siku pada elemen. Karakter `[ ]` pada elemen bisa dianggap tag gaya oleh *rich* dan hilang dari tampilan. Solusinya memakai `rich.markup.escape()` pada teks elemen.

4. Import tidak terpakai. `from typing import Any` tidak dipakai dan boleh dihapus.

5. Kompleksitas. Jumlah permutasi tumbuh sangat cepat (faktorial), sehingga membuat semua susunan membutuhkan waktu dan memori sebanding dengan jumlahnya, yaitu O(P(n,r) × r).

7. Ringkasan Cepat dan Latihan

Ringkasan untuk dihafal

Konsep	Yang perlu diingat
Permutasi	Urutan penting; `math.perm`, `itertools.permutations`
Kombinasi	Urutan tidak penting; `math.comb`, `itertools.combinations`
rich	Mempercantik terminal: Panel, Table, Prompt
IntPrompt	Input angka dengan validasi otomatis
list comprehension	Membuat list ringkas dalam satu baris
chr(65+i)	Menghasilkan huruf A, B, C, ...
while True + break	Menu berulang sampai pengguna keluar
if __name__ == "__main__"	Menjalankan main() hanya jika file dieksekusi langsung

Latihan pemahaman

- Berapa P(5,3) dan C(5,3)? *(Jawab: 60 dan 10)*
- Mengapa P(n,r) selalu lebih besar atau sama dengan C(n,r)? *(Karena $P = C \times r!$)*
- Apa yang terjadi jika pengguna memasukkan r = 0? *(Ditolak dan diminta ulang karena r harus 1..n)*
- Mengapa dipakai `console.width` untuk menentukan jumlah kolom? *(Agar tabel menyesuaikan lebar terminal)*